A3-PS021-TC497

Sutra: Dialog Context and Safe Retry for AI Agent Chains

Ronel Abraham Mathew
ronelabraham@karunya.edu.in

Gavriel Stephen Elijah
gavrielstephen@karunya.edu.in

Dr. Salaja Silas
salaja_cse@karunya.edu

AIORI Hackathon 3

October 14, 2026

Problem Description

When a chain of AI agents and tools is cut mid-step, nobody can tell whether an action ran, and a retry repeats it.

Lost reply. A tool commits (for example a payment) but the reply never arrives. The caller cannot tell “never ran” from “ran, reply lost”.

Repeated action. The natural fix, retrying, looks like a new request, so the payment or booking happens twice.

No shared thread. Each hop in A2A and MCP tracks its own IDs, so nothing ties one user dialog together or says how far it got.

Target (PS-021). A clearly labelled experimental prototype with reproducible failure tests.

Example: Priya books a Tatkal ticket on a train. The payment goes through, the train enters a tunnel and the confirmation is lost. Her assistant retries, and she is charged twice.

Background

Agent requests now cross several protocols, and the protocol layer no longer carries continuity.

- A2A (agent-to-agent) and MCP (agent-to-tool) exchanges are chained: one user request can cross both.
- MCP SEP-2575 removed the initialize handshake and resumable streams. A dropped connection now cancels the request, and durable work must use the tasks primitive [1].
- Trace-ID propagation works in closed environments but not in open ones (IETF 126 BoF) [2].
- IETF agentproto is now a working group. Its charter covers dialog context (IDs plus lifecycle state) and recovery from failures of network paths and intermediaries; use-cases draft due 2026-12-31 [3].

IETF 126 BoF, 23 Jul 2026 (show of hands): forming a group 154 yes / 51 no; willing to implement 42 yes / 28 no. A participant suggested waiting for a proof of concept [2].

Literature & RFC Review

Existing work covers idempotency keys and context carriage separately; we found no cross-protocol measurement of both.

HTTP Idempotency-Key draft [4]. Client-chosen key; 409 for a retry still in flight; 422 for the same key with a different payload; completed retries replay the first result. Expired Internet-Draft.

MCP SEP-3182 [5]. Optional idempotencyKey on tools/call plus a tools.idempotency capability; same key with different arguments is rejected. Open proposal, status unconfirmed.

MCP SEP-414 [6]. Documents trace-context propagation in _meta, a precedent for carrying context there.

A2A. Already has a context ID shared across agents; the working group may define its semantics [2].

Gap. No published measurement found of which carrier (HTTP header, MCP _meta, A2A extension) delivers context and key intact under injected failures. This is our search, not proof none exists.

Proposed Solution

Sutra is a small layer that gives each dialog one shared context and makes a retried action return the saved result.

1. Dialog context. Conversation ID, hop, step and version, never the conversation content. Carried as an HTTP header and in MCP _meta; an A2A extension is a stretch goal.

2. Derived idempotency key. A keyed hash of context and step, so a replacement agent holding the same context recomputes the same key.

3. Durable operation store. At the effect-holder: atomic check-and-record, a retry returns the saved result, and the same key with a changed action is rejected.

4. IN_DOUBT state. After a lost reply, ask the effect-holder “did key K commit?”: return the stored result, or retry safely with the same key.

5. DialogChaos harness. Injects drops and crashes; compares no protection, a plain persisted random key, and Sutra.

Optimization Proposed

Target behaviour per failure scenario; DialogChaos will measure whether it holds.

Scenario	Without protection	With Sutra (target)
Reply lost after the payment commits	Retry runs the payment again	Retry carries the same key and gets the stored result
Agent crashes; a fresh copy takes over	New copy has no record and repeats the step	Replacement recomputes the same key from the context
Crash after the effect, before it is recorded	Outcome unknown; retried blindly	IN_DOUBT; query “did key K commit?” before any retry
Retry arrives while the original still runs	Two copies can both execute	Atomic check-and-record lets only one through
A carrier or gateway strips the context	Silent loss of continuity	Measured per carrier, so the loss is visible

Table shows design targets, not measured results.

Solution Architecture & Design

Visual: components, the Sutra adapters and the test lab that breaks the links on purpose.

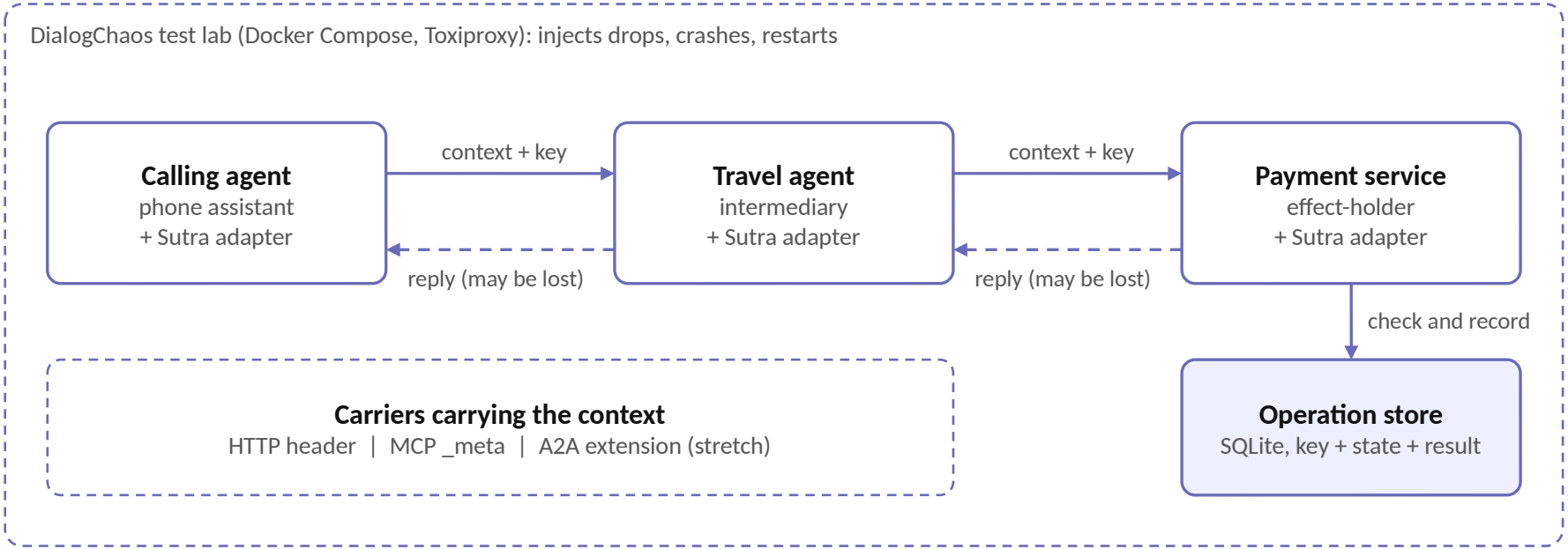


Figure: Sutra component architecture (planned prototype)

Data Flow Diagram

Overview: how a request, its key and its saved result move, including the lost-reply retry.

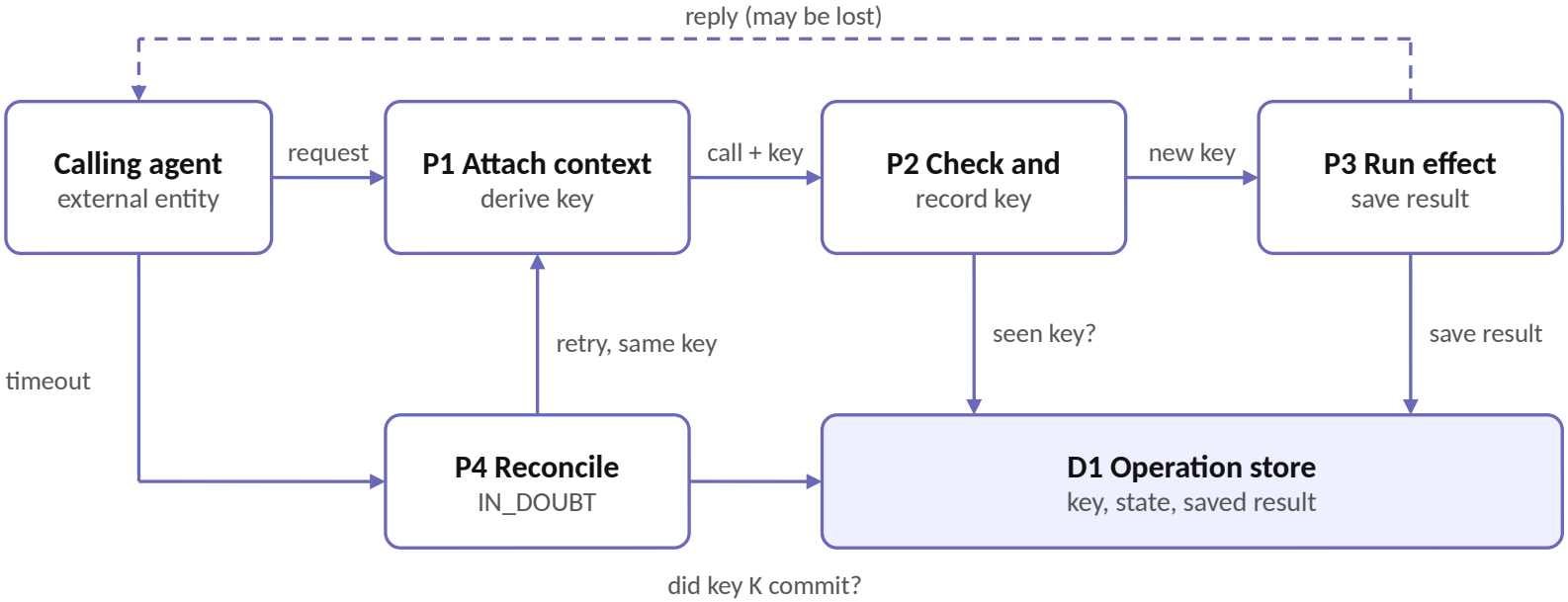
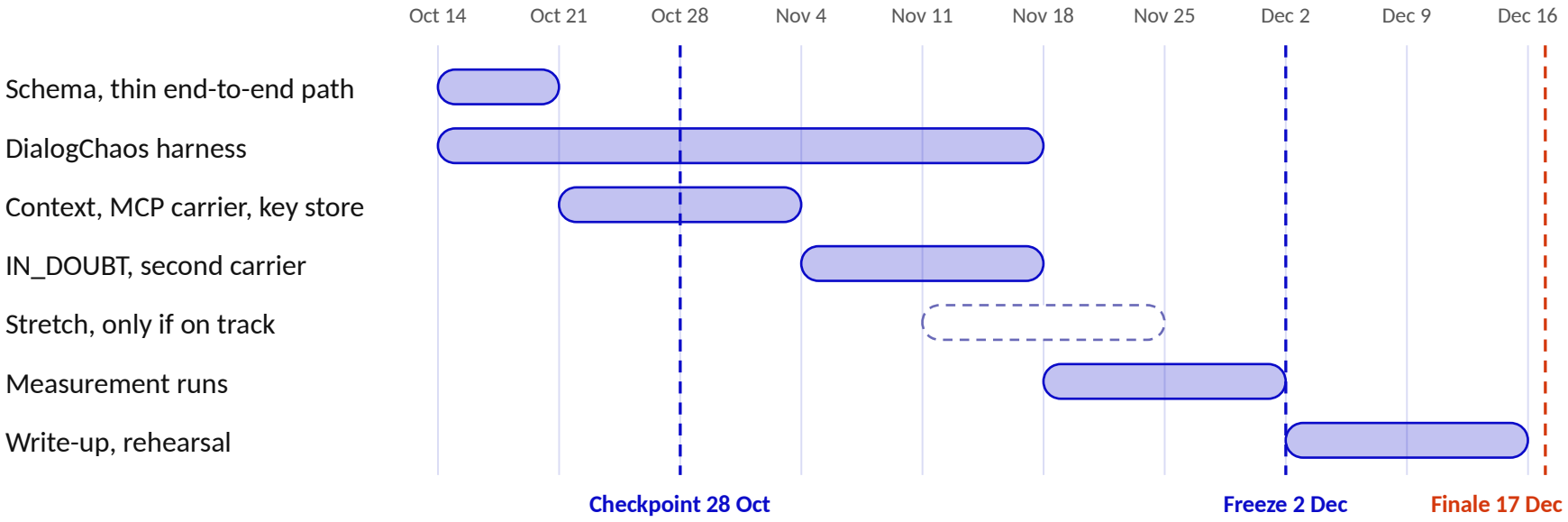


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Nine weeks to the 17 December finale, with a go/no-go checkpoint at week 2.



Method: weekly Friday demo to the faculty mentor; each failure scenario written as a test before its recovery logic; every change reviewed by another team member. At the checkpoint the thin path must show one charge with Sutra and two without.

Bibliography

[1] Model Context Protocol. “SEP-2575: Make MCP Stateless.” Final, Standards Track. https://modelcontextprotocol.io/seps/2575-stateless-mcp

[2] IETF. “Minutes: Agent Communication Protocols (agentproto) BoF.” IETF 126, Vienna, 23 July 2026. https://datatracker.ietf.org/meeting/126/materials/minutes-126-agentproto-202607230700-00

[3] IETF. “Agent Communication Protocols (agentproto)” working group page and charter (charter-ietf-agentproto-01). https://datatracker.ietf.org/wg/agentproto/about/

[4] J. Jena, S. Dalal. “The Idempotency-Key HTTP Header Field.” draft-ietf-httpapi-idempotency-key-header-07, 15 October 2025 (expired Internet-Draft). https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/

[5] Model Context Protocol. “SEP-3182: Request Idempotency.” Pull request #3182, open proposal; status not confirmed. https://github.com/modelcontextprotocol/modelcontextprotocol/pull/3182

[6] Model Context Protocol. “SEP-414: Document OpenTelemetry Trace Context Propagation Conventions.” Final, Standards Track. https://modelcontextprotocol.io/seps/414-request-meta

Thank You for Your Attention!